บวกเลขสองจำนวน (addTwo)

โจทย์ข้อนี้เป็นข้อตัวอย่างเพื่อทำความรู้จักการส่งโจทย์แบบฟังก์ชัน ให้เขียนฟังก์ชัน addTwo ที่รับจำนวนเต็ม a และ b แล้วคืนค่าผลบวก $a + b$ กลับไป ผู้เรียกฟังก์ชันจะเป็นโปรแกรมตรวจซึ่งเรียกซ้ำหลายครั้งด้วยค่า a, b ที่ต่างกัน

ข้อสังเกต: ค่า a และ b เป็นชนิด `int` แต่ผลบวกอาจมีค่ามากกว่าที่ `int` เก็บได้ ฟังก์ชันจึงมีชนิดผลลัพธ์เป็น `long long`

ทำความเข้าใจการส่งแบบฟังก์ชัน (อ่านก่อน)

โจทย์ปกติให้คุณเขียนโปรแกรมครบทั้งหมด คือมี `main` อ่านข้อมูลด้วย `cin` และพิมพ์ผลด้วย `cout` ส่วนโจทย์แบบฟังก์ชันให้คุณเขียนเพียงฟังก์ชันเดียว ส่วน `main` ผู้จัดเขียนไว้ให้แล้วในไฟล์ `grader.cpp` ซึ่งจะอ่านข้อมูลทดสอบ เรียกฟังก์ชันของคุณ แล้วพิมพ์ผลลัพธ์เอง

ไฟล์	ใครเป็นคนเขียน	หน้าที่
`add_two.h`	ผู้จัด (ห้ามแก้ไข)	ประกาศชื่อและชนิดของฟังก์ชันที่ต้องเขียน (prototype)
`add_two.cpp`	นักเรียน (ไฟล์ที่ส่ง)	เขียนตัวฟังก์ชัน `addTwo` ลงในไฟล์นี้
`grader.cpp`	ผู้จัด (อยู่ในระบบตรวจ)	มี `main` อ่านข้อมูล เรียก `addTwo` แล้วพิมพ์ผลลัพธ์

ตัวอย่างหน้าตาของ `grader.cpp` (ฉบับย่อ) นักเรียนไม่ต้องเขียนไฟล์นี้:

```cpp
#include "add_two.h"
#include <iostream>

int main() {
    int Q;
    std::cin >> Q;
    for (int t = 0; t < Q; ++t) {
        int a, b;
        std::cin >> a >> b;
        std::cout << addTwo(a, b) << "\n";   // เรียกฟังก์ชันของนักเรียน
    }
    return 0;
}
```

ไฟล์โครง `add_two.cpp` ที่แนบมากับโจทย์ (ยังไม่ได้เขียนอะไร):

```cpp
#include "add_two.h"

long long addTwo(int a, int b) {
    return 0;
}
```

ให้แก้ไขตัวฟังก์ชันเท่านั้น เช่น เปลี่ยนเป็น `return a + b;` แต่ต้องระวังช่วงของ `int` ในปัญหาย่อยที่ 3

ขั้นตอนการทำโจทย์แบบฟังก์ชัน

1. ดาวน์โหลดไฟล์แนบจากหน้าโจทย์: `add_two.cpp`, `add_two.h` และ `attachment.zip`
2. เปิดไฟล์ `add_two.cpp` แล้วเขียนตัวฟังก์ชัน ห้ามเปลี่ยนบรรทัดชื่อฟังก์ชัน

3. ทดลองในเครื่อง: แตก `attachment.zip` นำ `grader.cpp` ตัวอย่างมาไว้โฟลเดอร์เดียวกัน แล้วคอมไพล์สองไฟล์พร้อมกันด้วย `g++ -std=c++17 -O2 grader.cpp add_two.cpp -o add_two` จากนั้นรัน `./add_two < sample/sample1.in`
4. ส่งเฉพาะไฟล์ `add_two.cpp` ในระบบ (ชื่อไฟล์ต้องตรงตามที่กำหนด)
5. ระบบจะคอมไพล์ไฟล์ของคุณรวมกับ `grader.cpp` ตัวจริง รันกับชุดทดสอบ แล้วให้คะแนนตามปัญหาย่อย

ตัวอย่างลำดับการทำงานเมื่อ grader อ่านข้อมูล `3 4` หนึ่งบรรทัด:

ลำดับ	ใครทำ	เกิดอะไรขึ้น
1	`grader.cpp`	อ่านค่า $a = 3$ และ $b = 4$
2	`grader.cpp`	เรียก `addTwo(3, 4)`
3	ฟังก์ชันของนักเรียน	คืนค่า 7
4	`grader.cpp`	พิมพ์ 7 แล้วขึ้นบรรทัดใหม่

ข้อควรระวังเมื่อส่งแบบฟังก์ชัน:

- ห้ามเขียน `main` ในไฟล์ที่ส่ง เพราะจะซ้ำกับ `main` ของ grader ทำให้คอมไพล์ไม่ผ่าน
- ห้ามอ่านข้อมูลเองด้วย `cin` หรือ `scanf` และห้ามพิมพ์คำตอบเองด้วย `cout` หรือ `printf` เพราะ grader ทำให้แล้ว ถ้าพิมพ์เพิ่มคำตอบจะผิด
- ห้ามแก้ชื่อ ชนิดพารามิเตอร์ หรือชนิดผลลัพธ์ของฟังก์ชัน และห้ามลบบรรทัด `#include "add_two.h"`
- ถ้าคอมไพล์ไม่ผ่านจะได้ 0 คะแนนทุกปัญหาย่อย

รายละเอียดการเขียนโปรแกรม

เขียนฟังก์ชันต่อไปนี้ตามที่ประกาศในไฟล์ `add_two.h` โดยไม่ต้องเขียน `main` และไม่ต้องอ่านหรือพิมพ์ข้อมูลทางอินพุตและเอาต์พุตมาตรฐาน

```
long long addTwo(int a, int b);
```

- `a`, `b`: จำนวนเต็มที่จะนำมาบวกกัน
- ค่าที่คืนกลับ: ผลบวก $a + b$ (ชนิด `long long`)

การส่งงาน: ดาวน์โหลดไฟล์แนบ `add_two.cpp` (ไฟล์โครง) และ `add_two.h` จากหน้าโจทย์ แล้วเขียนฟังก์ชันลงในไฟล์ `add_two.cpp` ให้เสร็จ จากนั้นส่งเฉพาะไฟล์ `add_two.cpp` ระบบจะคอมไพล์ไฟล์ของคุณพร้อมกับ `grader.cpp` ของผู้จัด ไฟล์ `attachment.zip` มีโปรแกรมตรวจตัวอย่างสำหรับทดลองในเครื่อง (`g++ -std=c++17 -O2 grader.cpp add_two.cpp -o add_two`)

ข้อสำคัญ: โปรแกรมตรวจเรียกฟังก์ชันนี้ Q ครั้งต่อกรณีทดสอบตามลำดับในข้อมูลนำเข้า ห้ามแก้ไขไฟล์ `add_two.h` และชื่อกับชนิดของพารามิเตอร์ต้องตรงกับที่ประกาศไว้ทุกตัวอักษร แต่ละกรณีทดสอบเริ่มในโปรเซสใหม่ การบวก $a + b$ ที่เป็น `int` ทั้งคู่จะคำนวณด้วยชนิด `int` ก่อน แล้วจึงนำผลไปใส่ชนิดอื่น ถ้าผลบวกเกินช่วงของ `int` จะได้ค่าผิด

ตัวอย่างการเรียกฟังก์ชัน

การเรียกฟังก์ชัน	ค่าที่คืนหรือผลที่ได้
`addTwo(-5, 2)`	คืน -3
`addTwo(0, 0)`	คืน 0
`addTwo(2000000000, 2000000000)`	คืน 4000000000

ตารางแสดงตัวอย่างการเรียกที่แยกจากกัน ฟังก์ชันของคุณต้องรับมือกับค่าอื่นตามข้อจำกัดด้วย

ข้อจำกัด

- $1 \le Q \le 100\ 000$ (จำนวนครั้งที่โปรแกรมตรวจเรียกฟังก์ชัน)
- $-2\ 000\ 000\ 000 \le a, b \le 2\ 000\ 000\ 000$
- เวลา 1 วินาที และหน่วยความจำ 256 MiB ต่อกรณีทดสอบ

ปัญหาย่อย

1. (30 คะแนน) $0 \le a \le 100$ และ $0 \le b \le 100$
2. (30 คะแนน) $-1\ 000 \le a \le 1\ 000$ และ $-1\ 000 \le b \le 1\ 000$
3. (40 คะแนน) ไม่มีข้อจำกัดเพิ่มเติม

แต่ละปัญหาย่อยคิดคะแนนแยกกัน จะได้รับคะแนนของปัญหาย่อยเมื่อผ่านกรณีทดสอบทุกกรณีในปัญหาย่อยนั้น

รูปแบบข้อมูลสำหรับโปรแกรมตรวจตัวอย่าง

โปรแกรมตรวจตัวอย่างอ่านข้อมูลจากอินพุตมาตรฐาน บรรทัดแรกเป็นจำนวนเต็มบวก Q คือจำนวนครั้งที่เรียกฟังก์ชัน จากนั้นเป็นข้อมูลของการเรียกแต่ละครั้งตามลำดับ รวม Q ครั้ง ดังนี้
แต่ละบรรทัดมีจำนวนเต็ม a และ b คั่นด้วยช่องว่าง

ค่าที่ฟังก์ชันคืนกลับ บรรทัดละหนึ่งค่า (ผลลัพธ์ที่ต่อไปนี้ใช้ฟังก์ชันที่ทำงานถูกต้อง)

ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

ข้อมูลนำเข้า 1	ผลลัพธ์ 1
1 3 4	7

$3 + 4 = 7$
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 1, 2, 3

ข้อมูลนำเข้า 2	ผลลัพธ์ 2
3 -5 2 0 0 2000000000 2000000000	-3 0 4000000000

ครั้งที่ 1: $-5 + 2 = -3$ · ครั้งที่ 2: 0 · ครั้งที่ 3: $2\ 000\ 000\ 000 + 2\ 000\ 000\ 000 = 4\ 000\ 000\ 000$ (เกินช่วงของ `int`)
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 3